

ProgressPath

Educational progress tracker

Project documentation

JavaFX • SQLite • MVC

Documentation date: 10 September 2026

Implementation: Minimal MVC desktop application

Build: Maven, Java 17 release, JavaFX 21

Persistence: SQLite database in `data/progresspath.db`

Contents

1 Overview	3
2 Scope and screens	3
3 Architecture	3
4 AI assistant integration	4
4.1 Request flow	4
4.2 Configuration and boundaries	4
4.3 Changes in this revision	4
5 Database design	5
5.1 Entity relationship view	5
5.2 Tables and constraints	5
5.3 Persistence rules	6
6 Design patterns used	6
7 Patterns considered but skipped	7
8 Core workflows and calculations	7
8.1 Plan and progress workflow	7
8.2 Assignment workflow	8
8.3 Focus workflow	8
9 Build, seed, and test	8
10 Known limits and next steps	8
11 Source map	9

1 Overview

ProgressPath is a local desktop application for planning academic work and checking whether actual progress follows a target schedule. A student can create study plans, organise courses and weighted chapters, record progress history, manage assignments, run focus sessions, and inspect plan-level and course-level risk summaries.

The current implementation is deliberately small but complete enough to demonstrate the main workflows. The JavaFX view is a single shell with eight tabs. The controller coordinates user actions, the model facade owns validation and calculations, repositories isolate SQL, and SQLite stores data across restarts.

2 Scope and screens

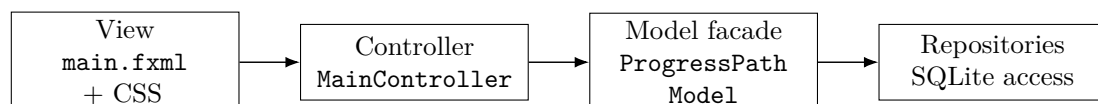
The current view is implemented in `main.fxml` and controlled by `MainController`. The screens and their responsibilities are:

Screen	Current features
Dashboard	Select an active plan; show actual and expected progress, status, overdue assignment count, course-risk table, and the next five active assignments.
Study plans	Create and edit plans, choose start and end dates, archive or restore a plan, and delete a plan with its dependent data.
Plan details	Select a plan, create or edit courses, create or edit chapters, assign chapter weights, set target dates, and review the course weight total.
Progress	Select a calculation strategy and course, update a chapter percentage, view progress history, and compare actual versus expected progress.
Assignments	Create or edit an assignment with course, title, description, due date, priority, and status; delete selected assignments.
Focus	Choose a course and optional chapter, start/pause/resume/complete/cancel a timer, add a note, and view saved sessions.
Analytics	Choose a plan and date range; show actual progress, expected progress, completed assignments, focus time, and an at-risk course report.
Assistant	Select an active plan, ask a study question, and receive an asynchronous Gemini Flash response based on a compact plan snapshot. The assistant does not edit stored data.

Charts, system notifications, and full-text assignment search are intentionally outside this minimal slice and are suitable future extensions.

3 Architecture

The application follows a compact Model–View–Controller flow:



- **View:** FXML declares controls and event names; CSS supplies the monochrome/blue application styling. The PDF documents the logic, not the screen colours.
- **Controller:** named handlers validate form-level concerns, call the model, and refresh tables and choices. It contains no SQL.
- **Model facade:** `ProgressPathModel` exposes plan, progress, assignment, focus, and analytics operations, validates domain rules, and notifies observers after mutations.

- **Persistence:** `DatabaseManager` opens SQLite connections with foreign keys enabled; repositories map rows to immutable Java records.
- **Headless seeding:** `SeedRunner` initializes the schema and invokes `DatabaseSeeder.seedDemoData()` without launching JavaFX.
- **AI provider:** the controller calls the `AssistantProvider` boundary; `GeminiFlashAssistantProvider` performs the network request without coupling Gemini code to JavaFX or SQLite.

4 AI assistant integration

The Assistant tab is an optional study-support screen. It uses the selected active plan to provide practical explanations, prioritisation suggestions, and short study plans. Existing progress calculations and database operations remain deterministic Java code; Gemini is used only to interpret the supplied snapshot and write guidance.

4.1 Request flow

1. The student selects an active study plan and enters a question.
2. The controller builds a compact context containing plan dates, actual and expected progress, courses, chapter progress/weights/targets, active assignments, and completed focus time.
3. A JavaFX Task calls `AssistantProvider` on a daemon worker thread so the interface remains responsive.
4. `GeminiFlashAssistantProvider` sends a `generateContent` request to the configured Flash model and returns the first text response.
5. The controller displays the answer or a clear configuration/network error; no AI response is written to SQLite.

4.2 Configuration and boundaries

The local project includes `.env.example`. Copy it to `.env` and set `GEMINI_API_KEY`; `.env` is ignored by Git. The provider accepts the documented `GOOGLE_API_KEY` or `GEMINI_API_KEY` environment variables (`GOOGLE_API_KEY` takes precedence), or the `-Dgoogle.api.key` / `-Dgemini.api.key` JVM properties. `GEMINI_MODEL` and `GEMINI_TIMEOUT_SECONDS` are optional overrides. The default model is `gemini-3.8-flash` and the default request timeout is 45 seconds.

The HTTP client follows the Gemini REST `models.generateContent` shape. It uses the `x-goog-api-key` header for authentication, keeping the key out of the request URL, and posts JSON containing a system instruction, one user content part, and a 512-token output limit.

If Google returns HTTP 403 with “Your project has been denied access”, the failure is project/key policy rather than application JSON. Create a new authorization key in an accessible Google AI Studio project, restrict it to the Gemini API, and replace the local value. The desktop client cannot override a denied project.

The app deliberately sends only the selected plan snapshot and the current question. It does not send the full database, prior conversation, or API key in the request body. For a distributed release, the provider call should move behind a server-side proxy so a key is not shipped in a desktop binary. The current minimal screen requires network access, does not persist chat history, and cannot guarantee that generated advice is correct; students should verify important academic decisions.

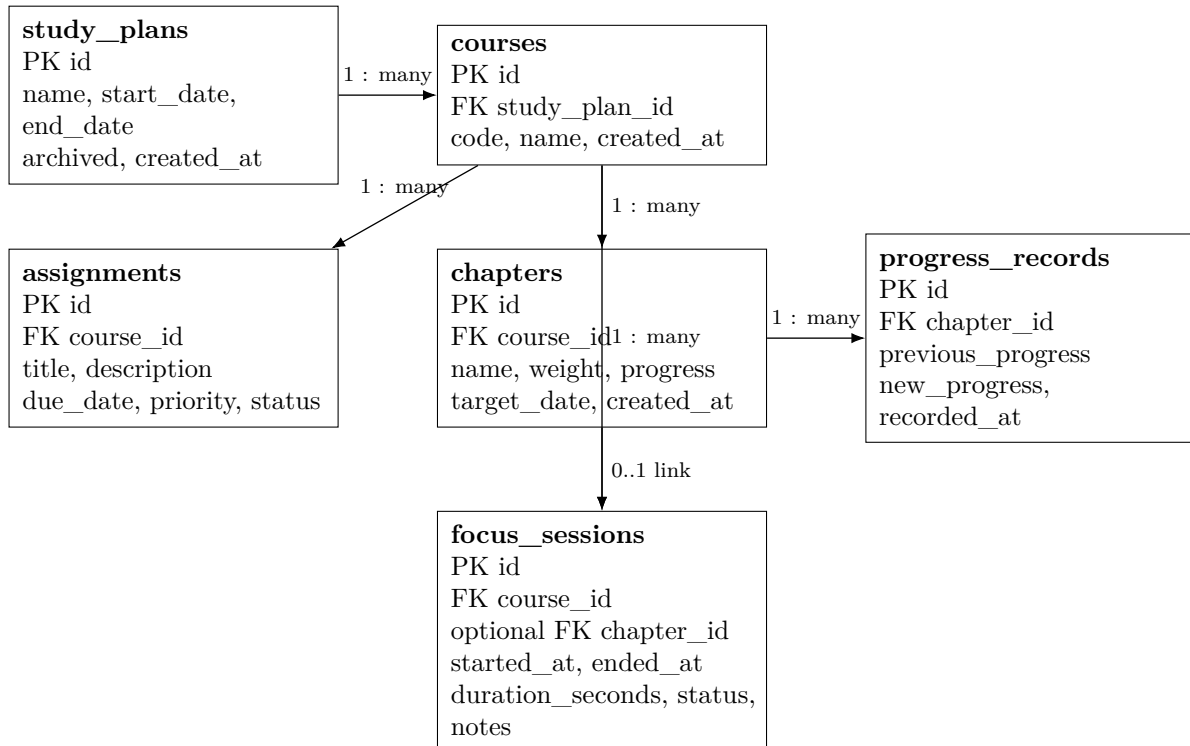
4.3 Changes in this revision

This revision adds the eighth, Assistant tab; the provider interface, Gemini Flash HTTP client, compact context builder, asynchronous JavaFX request handling, local `.env` loading, and the tracked `.env.example` template. Existing plan, course, progress, assignment, focus, analytics, persistence, and test workflows remain unchanged.

5 Database design

5.1 Entity relationship view

The database has six application tables. A plan owns courses; a course owns chapters and assignments; a chapter owns progress history; and focus sessions belong to a course with an optional chapter link.



5.2 Tables and constraints

The schema is created idempotently by `DatabaseSeeder.initialize()`. Existing databases receive an `archived` column migration when needed.

Table	Columns	Keys and rules
study_plans	id, name, start_date, end_date, archived, created_at	Primary key id; unique non-null name; archived is 0 or 1; end date must be after start date.
courses	id, study_plan_id, code, name, created_at	Primary key; plan foreign key with <code>ON DELETE CASCADE</code> ; unique (study_plan_id, code).
chapters	id, course_id, name, weight, progress, target_date, created_at	Course foreign key with cascade; unique (course_id, name); weight is greater than 0 and at most 100; progress is 0–100.
progress_records	id, chapter_id, previous_progress, new_progress, recorded_at	Chapter foreign key with cascade; both progress values are 0–100; indexed by chapter_id.
assignments	id, course_id, title, description, due_date, priority, status	Course foreign key with cascade; priority is LOW, MEDIUM, or HIGH; status is PENDING, IN_PROGRESS, COMPLETED, or CANCELLED; indexed by due date.

Table	Columns	Keys and rules
focus_sessions	id, course_id, nullable chapter_id, started_at, ended_at, duration_seconds, status, notes	Course foreign key with cascade; optional chapter foreign key with ON DELETE SET NULL; duration is non-negative; status is COMPLETED or CANCELLED; indexed by start time.

5.3 Persistence rules

- Every connection executes `PRAGMA foreign_keys = ON` and sets a three-second busy timeout.
- Domain validation supplements SQLite checks: plan dates are checked in the model, chapter weights are checked against the other chapters in the course, and text fields are trimmed and required where appropriate.
- Deleting a plan cascades to courses, chapters, assignments, focus sessions, and progress records through the foreign-key chain.
- The demo seeder uses one transaction, rolls back on failure, and returns a `SeedingReport`. If any plan already exists, it inserts nothing.
- Current demo data contains 11 study plans, 22 courses, 61 chapters, 30 progress records, 43 assignments, and 30 focus sessions.

6 Design patterns used

The following patterns are present in the current source. Each is used for a concrete separation or extension point rather than only for demonstration.

Pattern	Implementation	Why it is used
MVC	<code>main.fxml</code> (View), <code>MainController</code> (Controller), <code>ProgressPathModel</code> and records (Model)	Separates JavaFX event wiring from validation, calculations, and persistence. A screen can change without moving SQL or business rules into the view.
Repository	<code>StudyPlanRepository</code> , <code>CourseRepository</code> , <code>ChapterRepository</code> , <code>AssignmentRepository</code> , <code>FocusSessionRepository</code> , <code>ProgressRecordRepository</code>	Encapsulates SQL and row mapping. The model depends on simple domain operations instead of database statements, which keeps tests and a future storage replacement manageable.
Strategy	<code>ProgressCalculationStrategy</code> , weighted/equal policy classes, selected through <code>ProgressCalculator</code>	Makes the progress policy interchangeable. Weighted progress respects chapter weights; equal progress treats every chapter equally without changing the controller or model workflow.
State	<code>FocusSessionState</code> with Ready, Running, Paused, Completed, and Cancelled states; <code>FocusTimer</code> is the context	Keeps valid timer transitions next to the state that owns them. Invalid actions fail clearly, while timing details stay out of the JavaFX controller.
Observer	<code>ModelChangeListener</code> , <code>ModelChangeType</code> , and the controller subscription	A completed mutation notifies the interested UI. Tables, choices, dashboard metrics, and analytics refresh together without each editor knowing the internals of the other screens.

Pattern	Implementation	Why it is used
Provider Strategy	<code>AssistantProvider</code> with <code>GeminiFlashAssistantProvider</code>	Isolates the AI vendor and HTTP details from the controller. A second provider or a local test double can be added without changing the Assistant screen.

7 Patterns considered but skipped

These patterns are reasonable future options, but adding them to the current minimal slice would increase indirection without solving an active problem.

Pattern	Potential use	Reason it is skipped now
Factory Method / Abstract Factory	Construct different report objects such as weekly, monthly, and risk reports.	Analytics currently returns one parameterised <code>AnalyticsSummary</code> ; separate report products would be premature until report formats diverge.
Singleton	Make the database manager or application model globally accessible.	Global state would make isolated tests and multiple database paths harder. Dependencies are passed explicitly instead.
Command	Represent every button action as an object for undo, redo, or a work queue.	The current UI has immediate CRUD and timer actions, with no undo history or background command queue.
Decorator	Add optional cross-cutting behaviour such as logging, caching, or notifications around services.	The application has one local persistence path and no independent service implementations that need runtime composition.
Adapter	Wrap a second database, remote API, or external calendar behind the same interface.	SQLite is the only storage integration in this release; a wrapper would add code without a second incompatible API.
Event bus	Route many event types between numerous independent modules.	There are only a few screens and one model owner, so the small typed Observer interface is easier to trace and test.

8 Core workflows and calculations

8.1 Plan and progress workflow

1. Create a plan with a valid date range.
2. Add courses and chapters. The model validates required text, target dates, and the course's total chapter weight.
3. Choose weighted or equal chapter progress on the Progress screen.
4. Save a percentage for a chapter. The old value, new value, and timestamp are written to `progress_records` before the UI is notified.

5. The dashboard and analytics calculate expected progress from the plan start and each chapter target (or plan end when no target exists), then classify the variance as Ahead, On track, Slightly behind, or Significantly behind.

For weighted progress:

$$\text{progress} = \frac{\sum(\text{chapter progress} \times \text{chapter weight})}{\sum(\text{chapter weight})}.$$

Equal progress averages the chapter percentages. Both policies return zero when there are no chapters.

8.2 Assignment workflow

Assignments are linked to courses and ordered by due date, priority, and id. Overdue means the due date is before the selected day and the status is neither Completed nor Cancelled. The dashboard shows the first five active assignments; Analytics counts completed and overdue work for its selected date range.

8.3 Focus workflow

`FocusTimer.start()` moves Ready to Running. Running may pause, complete, or cancel; Paused may resume, complete, or cancel. Completing or cancelling records elapsed seconds, timestamps, status, and notes in `focus_sessions`. The controller uses a JavaFX timeline only to refresh the displayed clock; state transition rules remain in the State objects.

9 Build, seed, and test

Run these commands from the `ProgressPath` directory:

```
mvn test
mvn javafx:run
```

The application creates `data/progresspath.db` automatically. To load the current demonstration dataset:

Before using the Assistant tab, copy `.env.example` to `.env` and set `GOOGLE_API_KEY` or `GEMINI_API_KEY`. The application reads this file itself; Maven does not need a dotenv plugin. Never commit the resulting `.env` file.

```
mvn -q -DskipTests package
mvn -q exec:java -Dexec.mainClass=com.progresspath.app.SeedRunner
```

The seeder is safe to rerun: it reports `already_seeded=true` and inserts zero rows when the database already contains a plan. The current automated suite covers weighted and equal progress, persistence and validation, and the focus State lifecycle.

10 Known limits and next steps

The implementation is intentionally a minimal application. The following items are documented rather than hidden:

- Advanced assignment search and multi-field filtering are not in the current FXML.
- Analytics is tabular; chart components and exportable reports are future work.
- There is no operating-system reminder or notification service.
- Focus sessions are stored only when completed or cancelled; crash recovery is not yet persisted as an open-session record.
- Repository operations are intentionally straightforward; larger deployments could add transactions around multi-step plan setup and a service layer between the model facade and repositories.
- The Assistant tab depends on a configured Gemini key and network access. Responses are not stored, streamed, or used to mutate plans, courses, assignments, or progress.

11 Source map

The following locations are relative to `src/main/java/com/progresspath` unless noted otherwise.

Location	Responsibility
<code>app/</code>	JavaFX entry point and headless demo seeder.
<code>controller/</code>	FXML controller and screen event coordination.
<code>model/</code>	Immutable records, model facade, analytics values, and Observer event types.
<code>persistence/</code>	SQLite connection, schema/migration/seeding, and repositories.
<code>progress/</code>	Strategy interface and progress policies.
<code>focus/</code>	Focus timer context and State implementations.
<code>ai/</code>	Assistant provider boundary, Gemini Flash HTTP client, local configuration, and narrow response parsing.
<code>resources/fxml</code>	Eight-tab JavaFX View in <code>main.fxml</code> .
<code>resources/css</code>	Application styling in <code>progresspath.css</code> .
<code>src/test/java</code>	Unit tests for progress, persistence/validation, and focus states.